

R A P P O R T D E P R O J E T

CashControl

Application Android de gestion financière personnelle

Kotlin / Android / MVVM / Room

Formation : BTS SIO SLAM
Année scolaire : 2025 – 2026
Date : Mai 2026

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 2
Nom, prénom : CAREME Joey		N° candidat : 02051928866
Épreuve ponctuelle ☐	Contrôle en cours de formation ☒	Date : / /
Organisation support de la réalisation professionnelle Lycée Melkoir Garré — BTS SIO option 2 SLAM		
Intitulé de la réalisation professionnelle : CashControl		
Période de réalisation : De Mars 2026 à Mai 2026 Lieu : Cayenne Modalité : ☒ Seul(e) ☐ En équipe		
Compétences travaillées ☒ Concevoir et développer une solution applicative ☐ Assurer la maintenance corrective ou évolutive d'une solution applicative ☒ Gérer les données		
Conditions de réalisation¹ (ressources fournies, résultats attendus) Ressources : accès à l'environnement de développement Android Studio, documentation du référentiel BTS SIO option SLAM. Résultats attendus : une application Android fonctionnelle couvrant la gestion des transactions, du budget par catégorie, objectifs d'épargne et une calculatrice financière, une architecture propre et une base de données locale.		
Description des ressources documentaires, matérielles et logicielles utilisées² Matérielles : Ordinateur du candidat, smartphone Android pour les tests (ou émulateur Android Studio) Logiciels : Android Studio Hedgehog, Kotlin 1.9, SDK Android API 26 à 34		
Modalités d'accès aux productions³ et à leur documentation⁴ Android Studio Hedgehog sur le poste du candidat.		

¹ En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

² Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

³ Conformément au référentiel du BTS SIO « Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des »

Épreuve E5 - Conception et développement d'applications (option SLAM)

ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)**Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs**

CashControl est une application Android native développée en Kotlin, permettant à un utilisateur de gérer ses finances personnelles. Elle repose sur une architecture MVVM (Model-View-Model) avec localisation par pièce/salle.

Fonctionnalités principales :

- Tableau de bord : suivi des recettes, et soldes en temps réel
- Gestion du budget par catégorie avec alertes à 80% et 100%
- Objectifs d'épargne avec retrait et dépôts validés
- Calculatrice financière (TVA 20% / 8,5% DOM, prix HT, salaire net)

Travaux :

- Conception de la base de données (3 entités : Transaction, BudgetCatégorie, ÉconomiesOblige)
- Développement des DAOs, Référentiel, ViewModels et Fragments
- Mise en place des notifications push Android
- Audit et correction de 12 anomalies (2 bugs critiques, 6 importants, 4 améliorations)

interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve. ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

⁴ Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

1. Introduction & Contexte

1.1 Présentation du projet

CashControl est une application Android native de gestion financière personnelle, développée en Kotlin dans le cadre d'une formation en informatique.

L'application repose sur une architecture MVVM (Model-View-ViewModel), recommandée par Google pour le développement Android moderne. La persistance des données est assurée localement via Room (surcouche SQLite), sans aucune dépendance à un serveur distant. L'utilisateur dispose d'un accès immédiat à ses données, même hors connexion.

1.2 Objectifs

- Suivre les revenus et dépenses au quotidien grâce à un historique de transactions.
- Planifier et respecter un budget mensuel par catégorie de dépenses.
- Gérer des objectifs d'épargne avec visualisation de la progression.
- Calculer des formules financières courantes : TVA (Métropole et DOM), prix HT, salaire net estimé.
- Alerter l'utilisateur en temps réel lorsqu'un budget est proche d'être dépassé (80% et 100%).
- Garantir la sécurité et la cohérence des données via des validations côté ViewModel.

1.3 Technologies utilisées

Stack technique

Langage : **Kotlin** | Architecture : **MVVM**

Base de données : **Room / SQLite (cashcontrol_db)**

UI : **XML Layouts, ViewBinding, Material Components 1.11.0**

Asynchronisme : **Coroutines Kotlin + viewModelScope**

Build : **Gradle + KSP** (Kotlin Symbol Processing, remplace kapt)

2. Fonctionnalités Détaillées

2.1 Tableau de bord (DashboardFragment)

Écran principal de l'application. Il affiche en temps réel le solde du compte, le total des revenus et le total des dépenses, calculés directement depuis la base de données Room via des requêtes agrégées (SUM). La liste des transactions récentes est affichée dans un RecyclerView avec DiffUtil pour des animations de mise à jour fluides.

- Affichage : solde, revenus totaux, dépenses totales.
- Liste des transactions : description, catégorie, montant coloré (vert/rouge), date.
- Ajout d'une transaction via un FAB ouvrant un dialogue contextuel.
- Sélection du type Revenu / Dépense avec retour visuel sur les boutons.
- Catégories chargées dynamiquement depuis la base (synchronisées avec l'onglet Budget).
- Suppression par appui long avec confirmation AlertDialog (irréversible).

2.2 Gestion du budget (BudgetFragment)

L'onglet Budget permet de créer des catégories de dépenses avec une limite mensuelle. Pour chaque catégorie, l'application calcule le total dépensé en interrogeant la table des transactions filtrées par catégorie, et affiche une barre de progression colorée.

- Affichage : nom, emoji, montant dépensé, limite, pourcentage d'utilisation.
- Récapitulatif global : total dépensé / total budgété / montant restant.
- Création, modification et suppression de catégories avec confirmation.
- Notification push à 80% et 100% du budget atteint, avec anti-doublon via SharedPreferences.

❑ Synchronisation Dashboard ↔ Budget

Les catégories créées dans l'onglet Budget sont automatiquement disponibles dans le dialogue d'ajout de transaction du Dashboard. Les deux écrans partagent la même source de données (Room LiveData).

2.3 Objectifs d'épargne (SavingsFragment)

L'onglet Épargne permet de définir des objectifs financiers (voyage, achat, fonds d'urgence...) avec un montant cible. L'utilisateur peut effectuer des dépôts et des retraits sur chaque objectif, visualiser la progression via une barre colorée, et suivre le total épargné sur l'ensemble des objectifs.

- Création d'un objectif : nom, emoji, montant cible.
- Dépôt : le montant s'ajoute au solde, plafonné au montant cible.
- Retrait : validé contre le solde disponible — un retrait supérieur au solde est refusé avec message d'erreur explicite.
- Suppression avec confirmation AlertDialog.

2.4 Calculatrice financière (CalculatorFragment)

La calculatrice intègre les opérations arithmétiques classiques ainsi que trois formules financières courantes. Un sélecteur de taux de TVA permet de basculer entre deux régimes fiscaux.

- Opérations : addition, soustraction, multiplication, division, pourcentage, changement de signe.
- Formule TVA (+ TVA) : calcul du prix TTC depuis le prix HT, au taux sélectionné.
- Formule HT (Prix HT) : calcul du prix HT depuis le prix TTC, au taux sélectionné.
- Formule Salaire Net : estimation du net à partir du brut (coefficient 0,78).

Taux de TVA	Zone applicable	Exemple
20%	France métropole (Alsace, Moselle inclus)	100 € HT → 120 € TTC
8,5%	Départements d'outre-mer (Martinique, Guadeloupe, Guyane, La Réunion, Mayotte)	100 € HT → 108,50 € TTC

2.5 Architecture des fichiers

Fichier / Dossier	Rôle
FinanceApplication.kt	Classe Application — initialisation du canal de notifications
MainActivity.kt	Activité principale — NavController + BottomNavigationView
data/entity/	Entités Room : TransactionEntity, BudgetCategoryEntity, SavingsGoalEntity
data/dao/	Interfaces DAO Room : requêtes SQL typées (SUM, INSERT, DELETE...)
data/db/FinanceDatabase.kt	Singleton Room, callback onCreate pour les données par défaut
data/repository/FinanceRepository.kt	Point d'accès unique aux DAOs — découple UI et base de données
viewmodel/DashboardViewModel.kt	Statistiques financières + LiveData transactions et catégories
viewmodel/BudgetViewModel.kt	Dépenses par catégorie + logique de notifications budget
viewmodel/SavingsViewModel.kt	CRUD objectifs d'épargne + validation dépôt/retrait
ui/dashboard/DashboardFragment.kt	Interface tableau de bord
ui/budget/BudgetFragment.kt	Interface gestion du budget
ui/savings/SavingsFragment.kt	Interface objectifs d'épargne
ui/calculator/CalculatorFragment.kt	Interface calculatrice financière

Fichier / Dossier	Rôle
adapter/TransactionAdapter.kt	RecyclerView transactions (ListAdapter + DiffUtil)
adapter/BudgetAdapter.kt	RecyclerView catégories budget
adapter/SavingsAdapter.kt	RecyclerView objectifs d'épargne (ListAdapter + DiffUtil)
utils/FormatUtils.kt	Formatters centralisés (devise et date) — évite la duplication
notifications/NotificationHelper.kt	Création et envoi des notifications push budget
notifications/BudgetNotificationReceiver.kt	BroadcastReceiver — prêt pour usage avec AlarmManager

3. Qualité du code

3.1 Bonnes pratiques appliquées

L'application respecte les recommandations officielles Android (Android Architecture Components) :

- Flux de données unidirectionnel : Room → Repository → ViewModel (LiveData) → Fragment.
- Pas d'accès direct à la base de données depuis les Fragments.
- ViewBinding activé sur tous les Fragments avec libération dans onDestroyView() (pas de fuite mémoire).
- ListAdapter avec DiffUtil pour les RecyclerViews — animations fluides et performances optimales.
- Singleton thread-safe pour FinanceDatabase avec double-checked locking.
- Coroutines sur Dispatchers.IO pour toutes les opérations de base de données.

✓ Architecture validée

Le projet suit le patron MVVM recommandé par Google. Chaque couche a une responsabilité unique et bien définie, ce qui facilite la maintenance, l'évolution et les tests unitaires.

3.2 Gestion des notifications

Le système de notifications budget est déclenché par le BudgetViewModel lors du recalcul des dépenses par catégorie. Deux seuils sont définis : 80% (avertissement) et 100% (dépassement). Un mécanisme anti-doublon basé sur SharedPreferences empêche l'envoi répété de la même alerte. Les flags sont réinitialisés automatiquement lorsque le budget redescend sous le seuil de 80%.

□ Point d'attention — Permission notifications Android 13+

Sur Android 13 (API 33) et supérieur, la permission POST_NOTIFICATIONS doit être accordée explicitement par l'utilisateur. L'application la demande au démarrage via `ActivityResultContracts.RequestPermission()`. Sans cette permission, aucune notification ne sera affichée.

4. Conclusion & Perspectives

4.1 Bilan du projet

CashControl v1.0 constitue une version stable, fonctionnelle et corrigée de l'application de gestion financière personnelle. Le renommage complet (FinanceApp → CashControl, com.financeapp → com.cashcontrol) a été réalisé sur l'ensemble des fichiers Kotlin, XML, Gradle et de navigation.

Les douze anomalies identifiées lors de l'audit — dont deux bugs critiques empêchant l'insertion des données par défaut et exposant l'application à une boucle infinie — ont toutes été corrigées. L'application a également été enrichie de fonctionnalités nouvelles : sélecteur de taux TVA adapté aux DOM, dialogues de confirmation avant suppression, et centralisation des utilitaires de formatage.

4.2 Améliorations envisageables

Axe	Amélioration envisagée
Fonctionnalités	Filtrage et recherche dans l'historique des transactions
Fonctionnalités	Graphiques de répartition (camembert par catégorie, courbe mensuelle)
Fonctionnalités	Export des données en CSV ou PDF
Fonctionnalités	Rappels planifiés via AlarmManager (le BudgetNotificationReceiver est prêt)
Fonctionnalités	Gestion multi-comptes (courant, livret, cash)

4.3 Compétences mobilisées

- Développement mobile Android natif en Kotlin avec les Jetpack Architecture Components.
- Conception et modélisation d'une base de données relationnelle avec Room / SQLite.
- Application du patron d'architecture MVVM avec séparation stricte des responsabilités.
- Gestion de l'asynchronisme non-bloquant avec les coroutines Kotlin et viewModelScope.
- Mise en place de notifications push Android avec gestion des permissions (API 33+).
- Débogage et audit de code : identification et correction d'anomalies de logique, de concurrence et d'expérience utilisateur.
- Utilisation de KSP (Kotlin Symbol Processing) pour la génération de code Room à la compilation.

✓ Conclusion

CashControl est prêt à être importé dans Android Studio, compilé et déployé sur tout appareil Android 8.0 (API 26) ou supérieur. Avec les améliorations identifiées — notamment l'ajout de graphiques, l'export de données et le chiffrement de la base — l'application pourrait évoluer vers une solution complète de gestion financière personnelle à destination du grand public.